@stacks/connect

Authenticate users and interact with Stacks wallets in web applications.

The `@stacks/connect` package provides a simple interface for connecting web applications to Stacks wallets, enabling user authentication, transaction signing, and message signing capabilities.

Installation

```
>_ Terminal                                    npm yarn pnpm bun

 $ npm install @stacks/connect
```

Connection functions

connect

`connect` initiates a wallet connection and stores user addresses in local storage.

Signature

```
function connect(options?: ConnectOptions): Promise<ConnectResponse>
```

Parameters

Name	Type	Required	Description
`options`	`ConnectOptions`	No	Configuration o for connection

Examples

Basic connection

```
import { connect } from '@stacks/connect';

const response = await connect();
// Response contains user addresses stored in local storage
```

Connection with options

```
const response = await connect({
  forceWalletSelect: true,
  approvedProviderIds: ['LeatherProvider', 'xverse']
});
```

isConnected

`isConnected` checks if a wallet is currently connected.

```
import { isConnected } from '@stacks/connect';

if (isConnected()) {
  console.log('Wallet is connected');
}
```

disconnect

`disconnect` clears the connection state and local storage.

```
import { disconnect } from '@stacks/connect';

disconnect(); // Clears wallet connection
```

getLocalStorage

`getLocalStorage` retrieves stored connection data.

```
import { getLocalStorage } from '@stacks/connect';

const data = getLocalStorage();
// {
//   "addresses": {
//     "stx": [{ "address": "SP2MF04VAGYHGAZWGTEDW5VYCPDWWSY08Z1QFNDSN" }],
//     "btc": [{ "address": "bc1pp3ha248m0mnaevhp0txfxj5xaxmy03h0j7zuj2upg34mt
```

```
//   }
// }
```

Request method

request

`request` is the primary method for interacting with connected wallets.

Signature

```
function request<T extends StacksMethod>(
  options: RequestOptions | undefined,
  method: T,
  params?: MethodParams[T]
): Promise<MethodResult[T]>
```

Parameters

Name	Type	Required	Description
`options`	`RequestOptions`	No	Request configu options
`method`	`StacksMethod`	Yes	Method to call o wallet
`params`	`MethodParams[T]`	Depends on method	Parameters for t method

Request options

Name	Type	Default	Descrip
`provider`	`StacksProvider`	Auto-detect	Custom instance

Name	Type	Default	Descrip
`forceWalletSelect`	`boolean`	`false`	Force w modal
`persistWalletSelect`	`boolean`	`true`	Persist v selectio
`enableOverrides`	`boolean`	`true`	Enable c fixes
`enableLocalStorage`	`boolean`	`true`	Store ac locally
`approvedProviderIds`	`string[]`	All providers	Filter av

Examples

Basic request

```
import { request } from '@stacks/connect';

const addresses = await request('getAddresses');
```

Request with options

```
const response = await request(
  { forceWalletSelect: true },
  'stx_transferStx',
  {
    amount: '1000000', // 1 STX in microSTX
    recipient: 'SP3FGQ8Z7JY9BWYZ5WM53E0M9NK7WHJF0691NZ159'
  }
);
```

Wallet methods

getAddresses

`getAddresses` retrieves Bitcoin and Stacks addresses from the wallet.

```
const response = await request('getAddresses');
// {
//   "addresses": [
//     {
//       "address": "bc1pp3ha248m0mnaevhp0txfxj5xaxmy03h0j7zuj2upg34mt7s7e32q7
//       "publicKey": "062bd2c825300d74f4f9feb6b2fec2590beac02b8938f0fc042a342
//     },
//     {
//       "address": "SP2MF04VAGYHGAZWGTEDW5VYCPDWWSY08Z1QFNDSN",
//       "publicKey": "02d3331cbb9f72fe635e6f87c2cf1a13cdea520f08c0cc68584a96e
//     }
//   ]
// }
```

sendTransfer

`sendTransfer` sends Bitcoin to one or more recipients.

```
const response = await request('sendTransfer', {
  recipients: [
    {
      address: 'bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t4',
      amount: '1000' // Amount in satoshis
    },
    {
      address: 'bc1qxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh',
      amount: '2000'
    }
  ]
});
// { "txid": "0x1234..." }
```

signPsbt

`signPsbt` signs a partially signed Bitcoin transaction.

```
const response = await request('signPsbt', {
  psbt: 'cHNidP...', // Base64 encoded PSBT
  signInputs: [{ index: 0, address }], // Optional: specify inputs to sign
  broadcast: false // Optional: broadcast after signing
});
// {
//   "psbt": "cHNidP...", // Signed PSBT
//   "txid": "0x1234..." // If broadcast is true
// }
```

Stacks-specific methods

stx_transferStx

`stx_transferStx` transfers STX tokens between addresses.

```
const response = await request('stx_transferStx', {
  amount: '1000000', // Amount in microSTX (1 STX = 1,000,000 microSTX)
  recipient: 'SP3FGQ8Z7JY9BWYZ5WM53E0M9NK7WHJF0691NZ159',
  memo: 'Payment for services', // Optional
  network: 'mainnet' // Optional, defaults to mainnet
});
// { "txid": "0x1234..." }
```

stx_callContract

`stx_callContract` calls a public function on a smart contract.

```
import { Cl } from '@stacks/transactions';

const response = await request('stx_callContract', {
  contract: 'SP2C2YFP12AJZB4MABJBAJ55XECVS7E4PMMZ89YZR.hello-world',
  functionName: 'say-hi',
  functionArgs: [Cl.stringUtf8('Hello!')],
  network: 'mainnet'
});
// { "txid": "0x1234..." }
```

stx_deployContract

`stx_deployContract` deploys a new smart contract.

```
const response = await request('stx_deployContract', {
  name: 'my-contract',
  clarityCode: `(define-public (say-hi (name (string-utf8 50)))
    (ok (concat "Hello, " name "!"))
  )`,
  clarityVersion: '2', // Optional
  network: 'testnet'
});
// { "txid": "0x1234..." }
```

stx_signMessage

`stx_signMessage` signs a plain text message.

```
const response = await request('stx_signMessage', {
  message: 'Hello, World!'
});
// {
//   "signature": "0x1234...",
//   "publicKey": "02d3331cbb9f72fe635e6f87c2cf1a13cdea520f08c0cc68584a96e8ac1
// }
```

stx_signStructuredMessage

`stx_signStructuredMessage` signs a structured Clarity message following SIP-018.

```
import { Cl } from '@stacks/transactions';

const message = Cl.tuple({
  action: Cl.stringAscii('transfer'),
  amount: Cl.uint(1000000n)
});

const domain = Cl.tuple({
  name: Cl.stringAscii('MyApp'),
  version: Cl.stringAscii('1.0.0'),
  'chain-id': Cl.uint(1) // mainnet
});

const response = await request('stx_signStructuredMessage', {
  message,
  domain
});
// {
//   "signature": "0x1234...",
//   "publicKey": "02d3331cbb9f72fe635e6f87c2cf1a13cdea520f08c0cc68584a96e8ac1
// }
```

stx_getAccounts

`stx_getAccounts` retrieves detailed account information including Gaia configuration.

```
const response = await request('stx_getAccounts');
// {
//   "addresses": [{
//     "address": "SP2MF04VAGYHGAZWGTEDW5VYCPDWWSY08Z1QFNDSN",
//     "publicKey": "02d3331cbb9f72fe635e6f87c2cf1a13cd...",
//     "gaiaHubUrl": "https://hub.hiro.so",
//     "gaiaAppKey": "0488ade4040658015580000000dc81e3a5..."
//   }]
```

```
// }
```

Error handling

Handle wallet errors using standard Promise error handling patterns.

```
try {
  const response = await request('stx_transferStx', {
    amount: '1000000',
    recipient: 'SP3FGQ8Z7JY9BWYZ5WM53E0M9NK7WHJF0691NZ159'
  });
  console.log('Transaction successful:', response.txid);
} catch (error) {
  console.error('Transaction failed:', error);
}
```

Advanced usage

requestRaw

`requestRaw` provides direct access to wallet providers without compatibility features.

Signature

```
function requestRaw<T extends StacksMethod>(
  provider: StacksProvider,
  method: T,
  params?: MethodParams[T]
): Promise<MethodResult[T]>
```

Example

```
import { requestRaw } from '@stacks/connect';

const provider = window.StacksProvider;
const response = await requestRaw(provider, 'getAddresses');
```

Migration notes

Version 8.x.x implements SIP-030 wallet standards. For legacy JWT-based authentication, use version 7.10.1.